2023 19th International Conference on Distributed Computing in Smart Systems and the Internet of Things (DCOSS-IoT)

2023 19th International Conference on Distributed Computing in Smart Systems and the Internet of Things (DCOSS-IoT) | 979-8-3503-4649-7/23/$31.00 ©2023 IEEE | DOI: 10.1109/DCOSS-IOT58021.2023.00074

Intent-driven Distributed Applications Management over Compute and Network Resources in the Computing Continuum

Anastasios Zafeiropoulos*, Eleni Fotopoulou†, Constantinos Vassilakis†, Ioannis Tzanettis†,
Chiara Lombardo‡, Alesssandro Carrega ‡, Roberto Bruschi‡
*Institute of Communication and Computer Systems, National Technical University of Athens, Athens, Greece
†School of Electrical and Computer Engineering, National Technical University of Athens, Athens, Greece
‡CNIT S2N National Laboratory, Genoa, Italy
tzafeir@cn.ntua.gr, {efotopoulou, cvassilakis, gtzanettis}@netmode.ntua.gr
chiara@tnt-lab.unige.it, {alessandro.carrega, roberto.bruschi}@unige.it

***Abstract*—Intent-driven orchestration enables organizations to achieve greater automation, integration, and intelligence of their compute and network resources by defining their objectives and policies in a more abstract and holistic way, while letting the orchestration system handle the coordination, optimization, and adaptation of the resources in response to changing needs and conditions. The trend for the development of intent-driven orchestration mechanisms has been growing in recent years as the deployment of distributed applications in the computing continuum has become more complex and diverse. In this manuscript we detail a novel approach for the management of distributed applications in the computing continuum. A hierarchical decision making scheme is proposed where an entity in a specific level of the hierarchy has the responsibility for the management of the overall application or a part of it, without having the control of all the management actions. The control is distributed across various entities that have to collaborate towards some joint objectives. Both compute and network resources management is considered that may be provided by the same or different infrastructure providers.**

***Index Terms*—intent-driven orchestration, computing continuum, system of systems, distributed applications, network operating support system**

I. Introduction

Applications development for the computing continuum relies on the adoption of microservices-based development paradigms [1], [2], where application developers break down monolithic applications into small, independent services which can be developed, deployed, and scaled independently. Such applications are distributed in nature and may be deployed across heterogeneous infrastructure in the computing continuum, where resources reservation and management spans from Internet of Things (IoT) devices to edge and cloud computing nodes [3]. This approach to the development of distributed applications is combined with the advent of serverless computing platforms [4], where a cloud provider manages the infrastructure and the resources needed to execute and scale applications based on user's application demands.

One of the main challenges to be faced towards the development of distributed applications for the computing continuum regards the increased complexity introduced for their lifecycle management. Allocation of resources may take place across the infrastructure where resources potentially are managed by different entities (e.g., edge/cloud computing providers, network providers) and orchestration platforms (e.g., multi-cluster cloud computing platforms, edge computing platforms, network operating support systems); thus, making inherent the need for the deployment of collaboration schemes among them. Such platforms are not suitable per se for addressing the orchestration challenges of distributed applications, since they usually have the responsibility but not the control of the reserved resources across the continuum, nor the knowledge and authorization for proper horizontal scheduling of the various application parts. Furthermore, as applications become more distributed, the coherence of failures begins to decrease, while the distance between cause and effect increases as well. Novel orchestration mechanisms are required for enabling the synergy among different orchestration systems/platforms by generalizing and modelling their orchestration modules.

The design of such an orchestration framework can follow a "system of systems" approach [5], where a set of complex systems (orchestration modules in the various parts of the continuum) are managed by a large-scale concurrent and distributed system. A "system of systems" refers to a group of self-governing systems integrated into a larger system, resulting in the provision of distinctive capabilities. These individual systems work together to generate a collective behavior which they would not be able to achieve separately. Each system regards an orchestration component or platform which is responsible for the management of a part of the

This research has been partially funded from the European Union's Horizon Europe research and innovation projects NEPHELE (Grant agreement No 101070487) and 6GREEN (Grant agreement No 101096925).

2325-2944/23/$31.00 ©2023 IEEE
DOI 10.1109/DCOSS-IoT58021.2023.00074

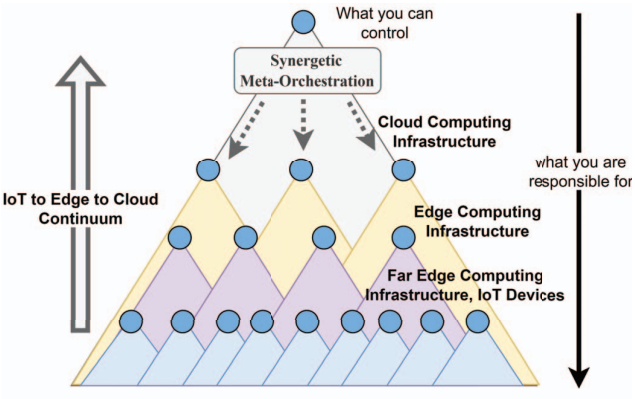

Fig. 1: System of systems approach in orchestration.

allocated resources in the continuum. A hierarchical tree structure is followed, where a system in a specific tree level is responsible for the management of the connected nodes in the lower tree levels, while it can directly control its children in the tree, as depicted in Figure 1. In this way, responsibilities for the orchestration of distinct parts of the application can be assigned on demand to different platforms.

Following a "system of systems" approach, to effectively handle the workloads managed by different microservices across the computing continuum, dictates the necessity to modify the way that deployment plans are produced and managed. One way to achieve so, involves the transitioning from creating plans tailored to allocating resources for each microservice to plans that prioritize the fulfillment of the objectives set by the application providers for the overall application. This shift in focus allows for more efficient management of workloads and resources in a way that is aligned with the goals and objectives of the application. Towards this direction, intent-driven orchestration mechanisms are emerging [6], [7]. Intent-driven orchestration mechanisms are a type of automation technology that enable the dynamic allocation and management of computing and networking resources based on the declared intent or goals of an application or service [8]. By automating the process of resource allocation and management, intent-driven orchestration mechanisms can help to reduce costs, improve performance, and ensure that resources are utilized in the most efficient way possible.

In the work presented in this manuscript, we detail an approach for intent-driven management of distributed applications that are deployed over compute and network resources in the computing continuum. The approach is designed taking into account emerging works on this area that mainly focus on intent-driven orchestration mechanisms for deployments in a specific part of the continuum. Our solution is introducing distributed intelligence, hierarchical decision making and autonomy characteristics while considering the need for collaboration among orchestration components in the various parts of the continuum.

The structure of the manuscript is as follows. In Section II we provide a short review of existing works related to intent-driven orchestration in the computing continuum. In Section III we present the representation of a distributed application graph where high level requirements can be declared for the nodes and links of the graph. Such requirements can be satisfied at deployment time by both computing and network management platforms. In Section IV we detail the designed approach for intent-driven orchestration of distributed applications, while Section V concludes the paper and refers to our future work plans.

II. Related Work and Motivation

Various solutions presented, support intent-driven orchestration mechanisms which mostly focus solely on compute or network resources allocation. For the compute part, focus is given on intent-driven orchestration for cloud-native applications deployments by considering a single provider infrastructure [6]. Architectural components responsible for planning, undertake the task of illustrating the translation of the provided intent to a deployment plan. Monitoring the application state during runtime and comparing it with the desired state triggers an evaluation to decide whether appropriate adjustments are required. Novel approaches arise considering the declarative control of distributed applications placement in the continuum [7]. In this case, the development of translation mechanisms becomes more challenging, since it is crucial to consider different ways of modeling the compute resources in the various parts of the continuum.

For the network part, the concept of intent-driven networking is introduced [9], [10] to translate a contextual definition of intent for Quality of Service (QoS) requirements to network services deployment, taking advantage of Software Defined Networking (SDN), Network Function Virtualization (NFV) and Machine Learning (ML) techniques [11]. To achieve so, abstractions for network functionalities are provided by network management systems. High level network policies can be mapped to such abstractions to activate network services to serve the defined intent, as it can be described through a Service Level Agreement (SLA). Such network services are usually made available through a network services repository or catalogue. There are several challenges to intent-based networking, such as the need for intent lifecycle management (intent description, intent assessment and validation, indent maintenance), and the need for automation in terms of translation mechanisms from high-level intent descriptions to low-level network configurations and actions [11].

In our approach, we envisage the support of human stakeholders in expressing their intents, based on the modeling of a distributed application as a graph and the declaration of a set of requirements, properties and restrictions in both node (application component) and link (communication between two application components) level. Based on a "system of systems" approach, following

orchestration actions are triggered in both the compute and network part of the infrastructure. The specification of open Application Programming Interfaces (APIs) for the exchange of information and the coordination of actions among the involved components is considered.

III. Declarative Distributed Applications Modeling

A distributed application is represented as an application graph consisting of application nodes and interconnection links. The nodes of the graph represent application components or consumed third-party services illustrating part of the business logic of the application. All of them are appropriately linked together in an overlay network, delivering the full application functionality in a synergetic way.

Declarative statements accompany the application graph in the form of tags (reflecting a value range for a metric or a set property) or simplified expressions. These statements address nodes and/or links or the whole application and aim to specify the application's objective and constraints regarding its deployment and operation to deliver the required results. An application's objective refers to an optional high-level goal set to pursuit when the application is deployed in allocated infrastructural resources and executed along its lifecycle (i.e. energy efficiency, real time response, high proximity to the users etc.). An application's constraints refer (1) to measurable requirements for each node and overlay link mostly specifying quantity and quality demands regarding required resources (i.e., computational power needed for an application component to run, overlay link bandwidth required between two application nodes, Quality of Service level expressing the value range for a set of network metrics, required for an overlay link etc.); (2) to properties enabling or setting required characteristics or functionalities for nodes and/or links (i.e., a secure overlay connection, autoscaling support for an application component etc.); and (3) to restrictions applying to single node or link or a group of nodes or links, expressed as simplified expressions (i.e., a required exclusive access to an application component, locality specifications for a node or a group of nodes, a collocation demand for a group of nodes etc.). Constraints set are further characterized as hard and soft if they should be satisfied by all means during the application's deployment and operation or if they should be treated in a best effort manner to be satisfied, respectively. The application's optional objective set is soft by nature since during deployment and operation there will be a pursuit for optimizing the associated objective function in the best possible way.

In Figure 2, a visual representation of an indicative application graph is provided. In Figure 2a, the main application components that compose the application graph are depicted, along with their constraints in terms of requirements, properties and restrictions. This intent-based declarative definition of the distributed application and associated overlay network is extended in Figure 2b where the application graph is augmented with the inclusion of services that illustrate

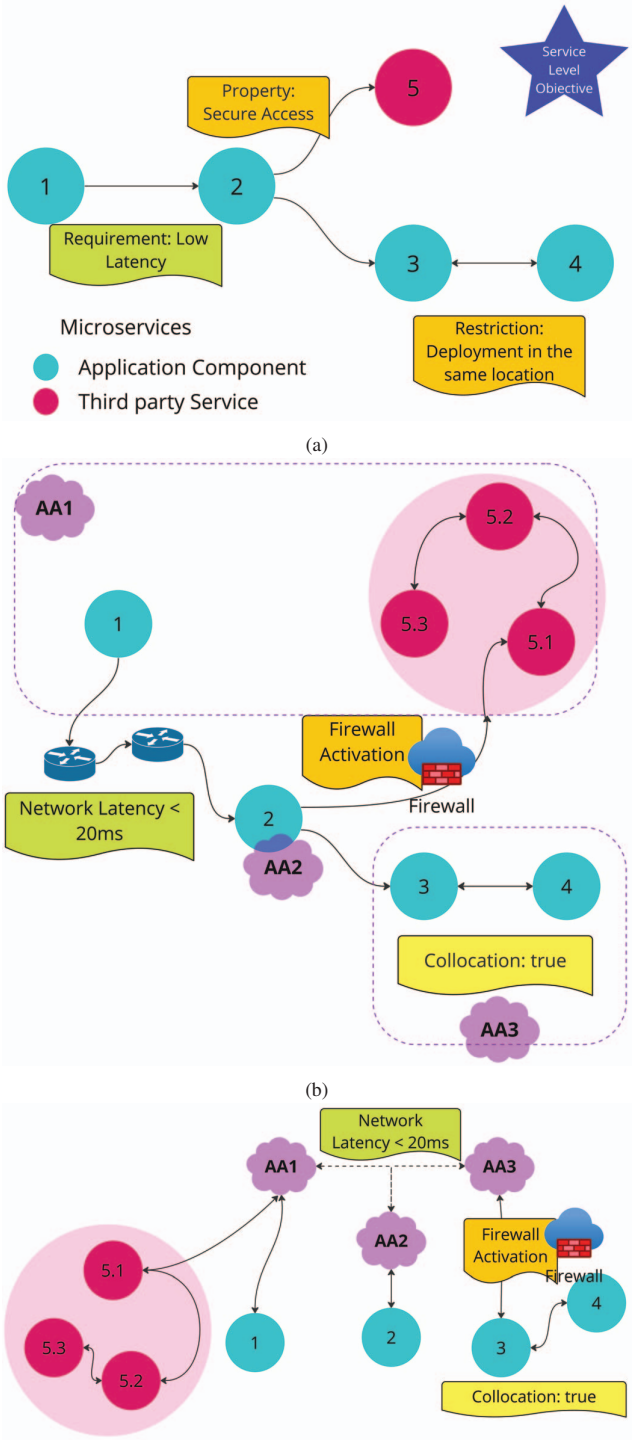

Fig. 2: Distributed application graph representation.

specified requirements, properties and restrictions. Third-party services chain structure is revealed while network-oriented functionalities are decomposed to network services required to be activated for the illustration of each functionality (e.g., activation of a network firewall, enforcement of a routing policy, activation of network observability mechanisms). Furthermore, each component in the augmented application graph is tagged with the administration authority (AA) that is responsible for the deployment and operation of each application component. In the computing continuum various providers (compute infrastructure providers, network infrastructure providers, services providers, edge and far edge infrastructure providers) collaborate in order to make accessible the offerings from each different administrative domain to the application developer and achieve through a negotiation and observability framework a synergetic deployment and orchestration of distributed applications not running under the control of a single authority. In Figure 2c the augmented application graph is partitioned per authority responsible for the deployment and operation of a group of nodes, while connection between authorities is represented with virtual links between Administrative Authorities service offering connection points. The latter are considered as the negotiation and communication points that illustrate bilateral agreed policies towards making it possible for a distributed application, partially operating in different administrative domains, to efficiently meet its objective and satisfy its constraints.

IV. System of Systems Approach for Intent-driven Orchestration

The proposed architectural approach for supporting intent-driven orchestration of distributed applications in the computing continuum is depicted in Figure 3. The top level entity is the Meta-orchestrator that is responsible to accommodate a deployment request for a distributed application. The deployment request includes the desired intent on behalf of the application provider in the form of declarative statements (objectives, constraints detailed as requirements, properties, restrictions). The set of declarative statements is formulated in the form of a Service Level Agreement (SLA) that has to be supported during the application lifecycle.

The main innovative characteristic of the proposed approach is that it jointly considers the interaction with a Multi-cluster Compute Manager and a Network Manager. These entities have to collaborate -under the supervision of the Meta-orchestrator- to support the deployment and the runtime management of the distributed application based on the defined SLA. Different business scenarios can be considered, depending on the interaction between application providers, cloud/edge computing providers and network providers (e.g., telecom operators). The existence of all or part of the aforementioned stakeholders may be applicable. For instance, if the networking part is considered as a commodity, both the computing and networking part may be managed by the same provider (e.g., the case where a multi Kubernetes

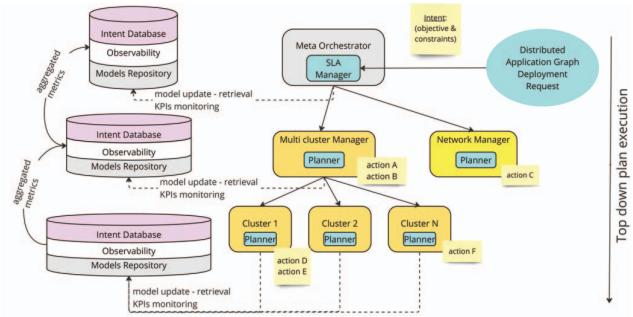

Fig. 3: Intent-driven orchestration in the computing continuum.

cluster is available where inter-cluster networking mechanisms are applied). In another case, the multi-cluster manager may belong to an edge/cloud computing provider and the network manager to a telecom provider. In each case, a "system of systems" is created and management responsibilities are assigned to different entities.

Continuous monitoring probes are activated for examining the state of the application graph in terms of the agreed SLA for achieving the desired intent. In case of deviations from the desired state, re-configuration actions take place and appropriate requests are triggered and sent to the Multi-cluster Compute Manager and/or the Network Manager.

Hierarchical decision making processes are applied based on the specification of orchestration control loops and relevant optimization process through a set of planners. To properly implement planning per hierarchical level, it is important to disassemble the high-level intent to a set of metrics that can be monitored at each level of the hierarchy. For instance, an intent for high availability and efficiency can be transformed to an SLA for managing 10 requests per second by a horizontally scalable application component with a service provision time that is less than 50ms. This request can be further translated in the enforcement of a specific scaling rule by the compute managers, as well as the provision of high priority to specific traffic flows in a network link (associated with a virtual application graph link) by the network manager. For the scaling part, metrics such as the average CPU and/or memory usage per container can be examined. For the network management part, the delay, jitter and packet loss metrics can be examined. Similarly, an intent for privacy can be translated in the deployment of a private 5G/6G network slice by the network manager. The translation rules from an intent to specific actions are made available to the intent database that exists in each level of the hierarchy.

To support the continuous monitoring of the various metrics, the development of cloud-native observability tools in the various levels is required. Data fusion of information coming from various signals (e.g., resource usage metrics, QoS metrics, software traces, logs) has to be considered [12]. Aggregation of metrics and production of composite metrics (e.g., Key Performance Indicators (KPIs)) is also needed, where aggregated metrics can be provided to higher levels

in the hierarchy for assisting decision making processes.

The collected data at each level is provided as input to the Planner component to guide any re-configuration in the runtime management of the distributed applications. Decision making can take place in any level in the hierarchy, while enforcement of actions can be applied in local level or in the lower levels of the hierarchy (e.g., the planner in a parent node can guide the operation of planners in the children nodes). Decision making is supported by the developed models that can be based -among others- on machine learning techniques (e.g., Reinforcement Learning, Multi-agent Reinforcement Learning, neural networks), rule-based management systems, and linear programming solvers. Specifically, the trained models can be available to the planners during deployment time (see the Models repository in Figure 3) and be further trained based on the data collected during the runtime phase of the distributed application. The models can be used for reactive planning (e.g., reaction to an event based on a rules-based management system), opportunistic planning (e.g., consume output of a solver with an objective to reach as much as close as possible to the desired objective) as well as proactive planning (e.g., based on forecasting mechanisms) [6]. Outcomes of the models in one level can provide input to models in higher level.

A. Network Resources Management

Enabling the deployment of cloud native, micro-service-based applications over a 5G and beyond (B5G) network requires to deal with the administrative domain separation between, on the one side, application orchestration and, on the other side, network orchestration, as the former is the responsibility of application service providers and cloud application developers, while the latter is operated by network/telecom providers. In this section, we provide details for the intent-driven network resources management in the case where this is supported by a network/telecom provider. In this scenario, the Operating Support System (OSS) is specifically conceived for simplifying and automating the management of distributed applications onto B5G infrastructures, by mostly hiding the complexity of the 5G environment to application developers and providers.

The interaction between the OSS and the meta-orchestrator can be done through the specification of open Application Programming Interfaces (APIs). It should be noted that various initiatives are considering this approach, such as the work in ETSI Multi-Access Edge Computing (MEC) for the specification of traffic management APIs [13] and the 3GPP Common API Framework for 3GPP Northbound APIs [14].

In this respect, we provide details where the proposed approach can be implemented by an open-source OSS that we have developed in our previous work [15]. The OSS is designed according to a highly modular architecture where all the software services are state-of-the-art cloud-native software. The OSS architecture is organized in a suite of five main software services, grouped into two main modules: the North-Bound OSS (NB-OSS) and the South-Bound OSS (SB-OSS).

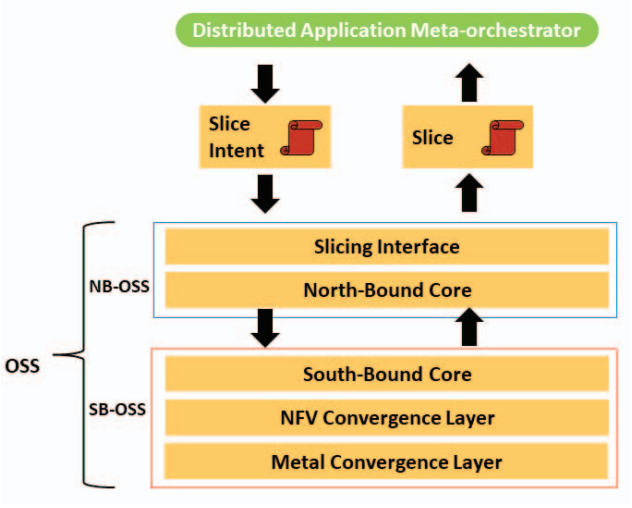

Fig. 4: Network Operating Support System.

The NB-OSS interacts with the meta-orchestrator. It manages network slice negotiations for distributed applications (Slicing Interface) and maintains metadata (e.g., coverage area served, operational capabilities, etc.) of one or multiple onboarded SB-OSS modules (North-Bound Core service). The SB-OSS is a chain of software services that can be selectively activated to gain access to various programmability levels, passing from a simple catalogue of available resources in case of no programmability, up to the complete terraforming of the physical infrastructure in case of full programmability.

The SB-OSS includes three "chained" services: the South-Bound Core service, the NFV Convergence Layer (NFVCL), and the Metal Convergence Layer (MetalCL). The South-Bound Core service is the only mandatory element in the SB-OSS, and it is devoted to process the slice instantiation/modification/de-instantiation requests and related resources. If the NFVCL and the MetalCL services are available, the SB Core can request to them the setup or the change of new or existing network slices/services and of infrastructure resources. The NFVCL manages the lifecycle of NFV services to provide suitable connectivity to distributed application components in a fully automated and zero-touch fashion. The MetalCL is the service dedicated to manage and terraform bare-metal resources (i.e., physical servers and hardware network equipment) to create IaaS/PaaS environments compliant with the 5G-platform needs.

The Slice Intent mechanism is responsible for requesting the creation of an application-aware network slice, considering the set of declared computational, networking (in terms of network services) and QoS requirements on behalf of the application provider. The deployment procedure of the network infrastructure to support the distributed application has three phases: (i) the transformation of the objectives and constraints of an intent into a network slice intent submitted to the Northbound API of the OSS, (ii) the realization of

the application slice by the OSS, and (iii) the deployment of the target application over the created slice by the meta-orchestrator.

The slice intent can describe at which locations the application components should be deployed, what policies and network requirements should be granted for these components for them to interact with each other or with external entities, and to meet desired performance criteria. The requirements for the network connectivity among two individual components are detailed in terms of delay, jitter, packet loss and throughput. The slice intent includes also User Equipment (UE) related metadata, considering the interaction between an application component with a UE (e.g., based on a specific QoS Class Identifier).

B. Compute Resources Management

We consider the management of compute resources across the continuum where a set of clusters are registered to a multi-cluster manager. The proposed approach is generic and can be applied in any type of orchestration technology; however, in our case we focus on the management of Kubernetes clusters. To support multi-cluster management, various open-source solutions can be adopted, such as Open Cluster Management [16], Karmada [17], Liqo [18], Razee [19]. Such tools can provide unified views of the resources managed by the various clusters. As detailed in the proposed approach in Section IV, hierarchical decision making is supported where the planner of the multi-cluster manager coordinates the planners of the various clusters across the continuum. Global decision making takes place at the multi-cluster manager planner, while the rest of the planners are able to take local decisions for their clusters.

The multi-cluster manager planner manages the placement of the distributed application components across the various clusters, aiming to achieve global optimization objectives (e.g., energy usage or cost optimization) and to introduce distributed intelligence characteristics. In case of edge computing scenarios, especially when combined with high mobility patters, the planner can guide live migration and workload offloading actions across the computing clusters to guarantee specific SLA parameters. It also supports the health check of the application based on the collection of monitoring feeds from the various clusters and service discovery mechanisms.

The planners in the various clusters coordinate local actions to achieve the desired intent state and introduce autonomy characteristics for local-decision making. The supported actions may regard scaling decisions (e.g., autoscaling mechanisms taking advantage of machine learning techniques [20]), lifecycle management of the application components (e.g., restart, self-healing actions upon specific events), enforcement of security policies and trusted computing mechanisms, enforcement of load balancing policies and parameterization of the monitoring probes (e.g., activation of probes or configuration of the data collection frequency).

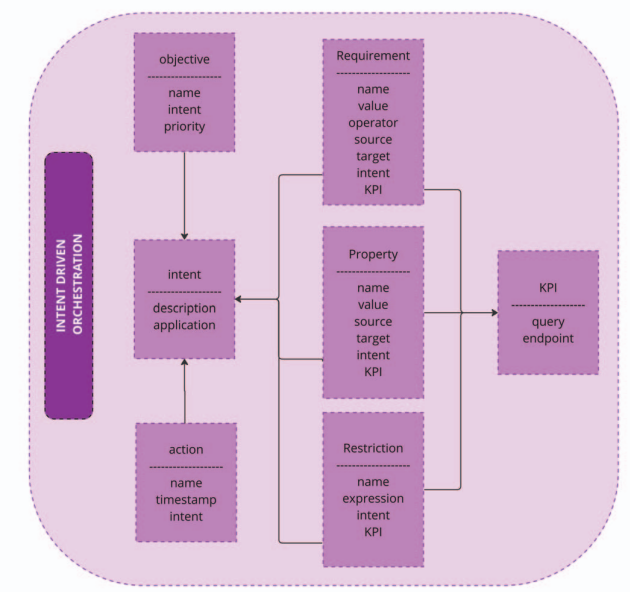

Fig. 5: Intent monitoring schema.

C. Data Management and Cloud-native Observability

Proper data representation schemas are essential for supporting intent-driven orchestration of cloud applications. They provide a common language for communication, facilitate automation, and enable the integration of diverse cloud resources and services through the exchange of information about the state of resources, dependencies, and other relevant factors. Different interpretations of the data schema may be present at the various levels of the hierarchy.

A relevant data schema to support the description of the intent and its mapping with metrics that are related to the defined requirements, properties and descriptions is provided in (Figure 5). A requirement can be represented in the form of an expression and be applied both in an application component (source) or a link between two application components (source and target of the link). The expression includes a metric (KPI), an operator or a range and the corresponding value(s) (e.g., a latency value for a link that is less than 20ms). In the case of properties, we follow a similar approach, however the value in this case can be a boolean or a compound metric (e.g., horizontal scaling support for an application component with a maximum of 10 replicas {autoscaling:true, max-pods:10}). In the case of restrictions, a rule-based expression is required (e.g., collocation for application component A and B) that can be interpreted by the planner, potentially through the support from an inference engine. The use of a Domain Specific Language (DSL) for a formal description of the requirements, the properties and the constraints that compose the intent can be considered; however, by recognizing that this may be hard in terms of expressivity for the end users [21].

To be able to monitor and evaluate the status of the current state per level of the hierarchy based on the aforementioned

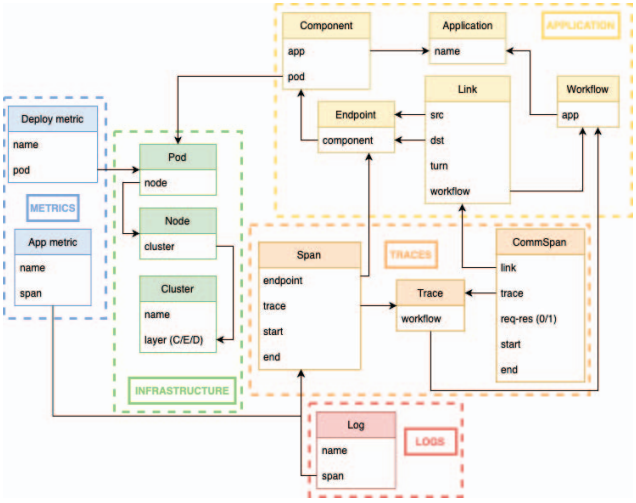

Fig. 6: Observability database schema.

metrics and expressions, modern observability solutions have to be adopted aligned with the emergence of cloud-native observability. In this way, queries can be applied by the various planners in the supported observability solutions, while the models in the relevant repository can be continuously trained and/or evaluated. To support cloud-native observability, a relational schema is proposed (Figure 6) as part of the architecture, extending the work at [12].

The schema is divided into five main sections, namely Application, Infrastructure, Metrics, Traces and Logs. A central aspect represented in the schema is the graph capturing the application components and their inter-dependencies. An application is composed of independent components and each component includes different API endpoints. In order to represent application workflows, i.e. sequences of API calls between components (endpoints in specific), we use the 'Link' model to define an API call from a source endpoint to a destination endpoint. Components are mapped to infrastructure nodes to provide a real-time view of the application deployment. The schema is especially designed to include real-time observability signals to the application graph construct. Traces represent workflow instances and provide the trace id for the corresponding computation and communication spans, which record the duration of the components' execution (Spans) and inter-communication (CommSpans) accordingly. Metrics and Logs are recorded and attributed to specific endpoints and application components.

V. Conclusions and Future Work

In the current manuscript we have detailed an approach for intent-driven orchestration of distributed applications across the computing continuum, considering the need for management of both the compute and network resources. The proposed approach is generic and can be adopted in various settings where the collaboration among application providers, cloud/edge computing providers and network providers may be required. In all cases, the overall system is decomposed in smaller collaborating systems under a hierarchical decision making scheme.

Based on the proposed approach, we plan to proceed to the implementation of the solution for intent-based orchestration in a multi-cluster setting. A Kubernetes-based orchestrator will interact with the presented OSS to support deployments of distributed applications over programmable compute and network infrastructure. The presented schemas for cloud-native observability will be adopted and supported through a developed software to support data fusion for various types of signals (metrics, logs, traces). The specification of a DSL for composing expressions to describe the desired intent will be also considered.

References

[1] K. Fu, W. Zhang, Q. Chen, D. Zeng, and M. Guo, "Adaptive resource efficient microservice deployment in cloud-edge continuum," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 8, pp. 1825–1840, 2022.

[2] S. Pallewatta, V. Kostakos, and R. Buyya, "Placement of microservices-based iot applications in fog computing: A taxonomy and future directions," 2023.

[3] S. Moreschini, F. Pecorelli, X. Li, S. Naz, D. Hästbacka, and D. Taibi, "Cloud continuum: The definition," *IEEE Access*, vol. 10, pp. 131876–131886, 2022.

[4] A. Mampage, S. Karunasekera, and R. Buyya, "A holistic view on resource management in serverless computing environments: Taxonomy and future directions," *ACM Comput. Surv.*, vol. 54, sep 2022.

[5] C. Keating, R. Rogers, R. Unal, D. Dryer, A. Sousa-Poza, R. Safford, W. Peterson, and G. Rabadi, "System of systems engineering," *Engineering Management Journal*, vol. 15, no. 3, pp. 36–45, 2003.

[6] T. Metsch, M. Viktorsson, A. Hoban, M. Vitali, R. Iyer, and E. Elmroth, "Intent-Driven Orchestration: Enforcing Service Level Objectives for Cloud Native Deployments," *SN Computer Science*, vol. 4, p. 268, Mar. 2023.

[7] J. Spillner, J. F. Borin, and L. F. Bittencourt, "Intent-based placement of microservices in computing continuums," in *Future Intent-Based Networking* (M. Klymash, M. Beshley, and A. Luntovskyy, eds.), (Cham), pp. 38–50, Springer International Publishing, 2022.

[8] A. Leivadeas and M. Falkner, "A survey on intent-based networking," *IEEE Communications Surveys Tutorials*, vol. 25, no. 1, pp. 625–655, 2023.

[9] K. Mehmood, K. Kralevska, and D. Palma, "Intent-driven autonomous network and service management in future cellular networks: A structured literature review," *Computer Networks*, vol. 220, p. 109477, 2023.

[10] L. Pang, C. Yang, D. Chen, Y. Song, and M. Guizani, "A survey on intent-driven networks," *IEEE Access*, vol. 8, pp. 22862–22873, 2020.

[11] A. Clemm, M. F. Zhani, and R. Boutaba, "Network management 2030: Operations and control of network 2030 services," *J. Netw. Syst. Manage.*, vol. 28, p. 721–750, oct 2020.

[12] I. Tzanettis, C.-M. Androna, A. Zafeiropoulos, E. Fotopoulou, and S. Papavassiliou, "Data fusion of observability signals for assisting orchestration of distributed applications," *Sensors*, vol. 22, no. 5, 2022.

[13] "ETSI Multi-Access Edge Computing (MEC) Traffic Management APIs," 2023. Available at https://www.etsi.org/deliver/etsi_gs/MEC/001_099/015/02.01.01_60/gs_mec015v020101p.pdf.

[14] N. D. Tangudu, N. Gupta, S. P. Shah, B. J. Pattan, and S. Chitturi, "Common framework for 5g northbound apis," in *2020 IEEE 3rd 5G World Forum (5GWF)*, pp. 275–280, 2020.

[15] R. Bolla, R. Bruschi, F. Davoli, C. Lombardo, and J. F. Pajo, "Multi-site resource allocation in a qos-aware 5g infrastructure," *IEEE Transactions on Network and Service Management*, vol. 19, no. 3, pp. 2034–2047, 2022.

[16] "Open Cluster Management Tool," 2023. Available at https://open-cluster-management.io/.

[17] "Karmada Open, Multi-Cloud, Multi-Cluster Kubernetes Orchestration," 2023. Available at https://karmada.io/.

[18] "Liqo Multi-Cluster Kubernetes Orchestration," 2023. Available at https://liqo.io/.
[19] "A multi-cluster continuous delivery tool for Kubernetes," 2023. Available at https://razee.io/.
[20] A. Zafeiropoulos, E. Fotopoulou, N. Filinis, and S. Papavassiliou, "Reinforcement learning-assisted autoscaling mechanisms for serverless computing platforms," *Simulation Modelling Practice and Theory*, vol. 116, p. 102461, 2022.
[21] M. Bezahaf, E. Davies, C. Rotsos, and N. Race, "To all intents and purposes: Towards flexible intent expression," in *2021 IEEE 7th International Conference on Network Softwarization (NetSoft)*, pp. 31–37, 2021.